



UNIVERSIDAD DE BURGOS

COMUNICACIÓN

CLIENTE-SERVIDOR

Alumno: Fernando Morán Fernández, con DNI 71378005N

Grado en Tecnologías Digitales para la Empresa

Profesor: Manuel Fernando Luque González

En Miranda de Ebro, a 09 de mayo de 2026

ÍNDICE

1.INTRODUCCIÓN	3
2.OBJETIVOS.....	3
3. MARCO TEÓRICO	3
4.DISEÑO Y METODOLOGÍA.....	3
5. IMPLEMENTACIÓN	4
6.RESULTADOS Y PRUEBAS.....	4
7.DISCUSIÓN.....	6
8.CONCLUSIONES	6
9.MANUAL DE EJECUCIÓN	6
10.REPOSITORIO GIT	7
11.BIBLIOGRAFÍA.....	7

1.INTRODUCCIÓN

La práctica simula un escenario real donde un dispositivo captura datos de una cámara térmica y lo transmite a una estación monitorización (cliente C++). La comunicación se establece mediante sockets TCP/IP, garantizando una transferencia de datos fiable y ordenada.

2.OBJETIVOS

- **Comprender el modelo cliente-servidor:** Establecer una conexión estable entre dos procesos independientes.
- **Uso de Sockets TCP/IP:** Implementar la creación, configuración y gestión de sockets bajo el estándar de Linux.
- **Gestión dinámica de memoria:** Utilizar estructuras como **vector** para almacenar flujos de datos de tamaño variable.
- **Persistencia de datos:** La información obtenida por red en archivos locales para su posterior análisis.
- **Automatización:** Crear scripts de Bash para simplificar el ciclo de compilación y ejecución.

3. MARCO TEÓRICO

Un socket es un punto final de una comunicación bidireccional entre dos programas que se ejecutan en la red. En la práctica se utiliza el protocolo TCP (Transmission Control Protocol), que se caracteriza por ser orientado a la conexión. Esto significa que antes de transmitir datos, el cliente y el servidor deben realizar un “saludo” (handshake) para asegurar que ambos están listos.

El flujo seguido ha sido:

- 1- Servidor: socket() -> bind() -> listen() -> accept().
- 2- Cliente: socket() -> connect() -> recv().

4.DISEÑO Y METODOLOGÍA

Se ha utilizado un diseño modular en C++ para separar la lógica de red de la lógica de aplicación. Se compone de lo estudiado el último día en clase:

- **Cliente c++:** Modularizado en tres archivos (.h, .cpp y main.cpp) para facilitar el mantenimiento y la legibilidad.
- **Servidor Python:** Simula la cámara térmica generando un array de 768 valores de tipo float.
- **Entorno de ejecución:** Se ha utilizado WSL para tener acceso a las librerías de sockets de Linux (<arpa/inet.h>).

5. IMPLEMENTACIÓN

Estructura de funciones C++

El cliente se ha estructurado en las siguientes funciones principales:

- **Crear_socket():** Inicializa el descriptor de archivo del socket usando `AF_INET` (IPv4) y `SOCK_STREAM` (TCP).
- **Configurar_direccion():** Utiliza la estructura `sockaddr_in` para definir la IP del servidor (127.0.0.1) y el puerto (12345). Se emplea `inet_pton` para convertir la IP a formato binario.
- **Recibir_datos():** Implementa un bucle `while` que invoca a `recv()`. Los datos se almacena en un `vector<float>` mediante un `push_back()`.
- **Guardar_datos:** Utiliza `ofstream` para crear el archivo `registro_temperaturas.txt` y recorrer el vector escribiendo cada valor.

Script de Automatización (Bash)

Se ha implementado `ejecutar.sh` para gestionar la compilación con `g++` comprobando el código de salida (\$?) para asegurar que el programa solo se ejecute si la compilación es exitosa.

6.RESULTADOS Y PRUEBAS

El programa cliente se conecta al servidor Python.

- Hacemos la ejecución de Python con `python3` `ServidorCamara.py` desde WSL.

```
fermoran@LAPTOP-SHJNG38H:/mnt/c/Users/ferki/Desktop/UNI/2_CUATRI/Sistemas_operativos/Práctica_3$ python3 ServidorCamara.py
Servidor escuchando en 127.0.0.1:12345
Conexión aceptada desde ('127.0.0.1', 56544)
Array de datos enviado al cliente.
```

- Ejecutamos la conexión con el servidor `./ejecutar.sh`.

```
fermorán@LAPTOP-SHJNG38H: /mnt/c/Users/ferki/Desktop/UNI/2_CUATRI/Sistemas_operativo...  
fermorán@LAPTOP-SHJNG38H:/mnt/c/Users/ferki/Desktop/UNI/2_CUATRI/Sistemas_operativo...  
s/Práctica_3$ nano Cliente_socket.cpp  
fermorán@LAPTOP-SHJNG38H:/mnt/c/Users/ferki/Desktop/UNI/2_CUATRI/Sistemas_operativo...  
s/Práctica_3$ ./ejecutar.sh  
Compilando...  
Compilación completada. Ejecutando cliente...  
=== INICIANDO CLIENTE DE TEMPERATURA ===  
Socket creado correctamente.  
Conexión establecida con el servidor  
Se han recibido 768 datos de temperatura correctamente.  
Datos guardados correctamente en el archivo: registro_temperaturas.txt  
  
Numeros de punto flotante recibidos:  
0  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
Conexión cerrada. Programa finalizado correctamente.  
fermorán@LAPTOP-SHJNG38H:/mnt/c/Users/ferki/Desktop/UNI/2_CUATRI/Sistemas_operativo...  
s/Práctica_3$
```

- Los resultados se guardan correctamente en **registro_temperaturas.txt**.

```
registro_temperaturas.txt
1  --- REGISTRO DE TEMPERATURAS ---
2  0
3  1
4  2
5  3
6  4
7  5
8  6
9  7
10 8
11 9
12 10
13 11
14 12
15 13
16 14
17 15
18 16
19 17
20 18
21 19
22 20
23 21
24 22
25 23
26 24
27 25
28 26
29 27
30 28
31 29
32 30
```

7.DISCUSIÓN

Hubo complicaciones entre el empaquetado de datos en Python (struct.pack) y la recepción en C++. Se ajustó el servidor para enviar floats reales (768f).

```
# Convertir los elementos de la lista a bytes
data_bytes = struct.pack('768f', *data_array)
```

8.CONCLUSIONES

La implementación de funciones modulares ha resultado en un código limpio, fácil de corregir errores y muy profesional. Además de la utilización de sockets.

9.MANUAL DE EJECUCIÓN

1- Preparación del servidor:

Abrir una terminal de Linux (WSL) y ejecutar:

`Python3 ServidorCamara.py`

2- Ejecución del cliente:

Abrir una segunda terminal de Linux en la misma carpeta y ejecutar:

`./ejecutar.sh`

El script se encarga de compilar main.cpp y cliente_socket.cpp y arrancar el cliente automáticamente.

10.REPOSITORIO GIT

El código fuente, los scripts y los resultados están disponibles en el siguiente repositorio:

https://github.com/Fer851/Practica_3_SO_Sockets

11.BIBLIOGRAFÍA

- <https://beej.us/guide/bgnet/html/#what-is-a-socket>
- <https://beej.us/guide/bgnet/html/#jumping-from-ipv4-to-ipv6>
- Apuntes de la asignatura: Sistemas operativos